

AI技术应用验证报告

AI技术应用验证报告

1. 验证目的

本次原型验证选择“AI 辅助需求文档生成与一致性检查”作为核心改进场景。

验证目标有三个：第一，测试 LLM 是否能根据现有软件过程材料生成结构化需求文档；第二，观察 AI 是否能补充人工容易遗漏的非功能需求、验收标准和一致性问题；第三，用实际过程数据对比传统人工整理方式与 AI 辅助方式在效率和质量上的差异。验证结果用于判断该场景是否适合纳入 Dyad 的过程改进方案。

2. 验证场景

验证对象选取 Dyad 项目的“自然语言生成应用原型”能力。

传统需求整理方式通常是开发者阅读项目背景、已有过程定义，然后手工提取功能需求、非功能需求和验收标准。这种方式可控，但耗时较长，容易漏掉安全、稳定性、跨平台兼容、失败恢复等非功能条件。

AI 辅助方式的流程是：将项目背景、过程定义和待分析场景输入 LLM，由模型生成结构化需求文档初稿；再让模型检查需求冲突、不可测试表述和 MVP 范围膨胀；最后由人类负责人审查并修订。

3. 工具选型

本次验证使用当前开发环境中的 Codex 作为真实 LLM 工具。工具形态为 IDE 内 AI 编码/文档协作助手，底层能力为 GPT-5 系列模型，能够读取本地文件、理解上下文、生成 Markdown 文档，并根据用户提示进行结构化输出。

4. 可复现验证过程

4.1 输入材料

原始过程设计文档。

4.2 执行提示词

本次使用的提示词如下：

1

你是一名软件需求分析师。请基于以下 Dyad 项目背景，生成一份“自然语言生成应用原型”功能的轻量需求规格。

2

3

项目背景：

4

Dyad 是一个本地运行的开源 AI 应用构建工具，采用 Electron + React + TypeScript 技术栈。它允许用户通过自然语言快速生成应用原型，支持界面、代码、交互的持续修改，本地预览，配置管理，项目导入导出和发布准备。项目过程强调最小可行过程，保留需求、设计、实现、测试、发布等核心工程活动，但避免重文档和重审批。

5

6

请输出：

7

1. 用户画像

8

2. 用户故事

9

3. 功能需求列表，使用 FR-编号

10

4. 非功能需求列表，使用 NFR-编号

11

5. 验收标准，使用 AC-编号

12

6. 需求一致性检查结果，包括冲突、不可测试表述、范围膨胀风险

13

7. MVP 建议范围

14

15

要求：

16

– 输出 Markdown

17

– 每条需求必须可以追踪到项目背景

18

– 验收标准必须可测试

19

– 不要编造与项目无关的业务功能

4.3 AI 执行过程记录

实际执行过程如下：

步骤	操作	输入	输出	用时
1	读取本地过程文档	Dyad 项目背景、过程定义	明确项目类型、技术栈、过程边界	约 1 分钟
2	向 Codex 提交需求生成提示词	上述 prompt	需求规格初稿	约 2 分 20 秒
3	要求 AI 进行一致性检查	需求初稿	冲突、不可测试项、范围风险	约 50 秒

4	人工快速审查	AI 输出结果	删除 1 条过度扩展需求，保留核心需求	约 6 分钟
5	整理验证报告	执行记录与对比数据	本报告	约 20 分钟

其中，真正由 AI 生成需求初稿和一致性检查的时间约为 3 分 10 秒。加上人工审查后，AI 辅助方式完成一版可用需求文档约需 9 分 10 秒。

5. AI 输出结果摘要

AI 生成的需求规格初稿包含以下内容。

5.1 用户画像

AI 将目标用户归纳为两类：一类是希望快速构建小型应用原型的开发者，另一类是熟悉自然语言表达但不熟悉完整工程配置的 AI 应用爱好者。该归纳与 Dyad 项目背景一致，没有引入无关行业场景。

5.2 功能需求样例

编号	AI 生成需求
FR-01	用户可以输入自然语言应用描述，系统生成项目结构和初始代码。
FR-02	系统应展示生成计划，包括将创建的页面、组件、数据结构和依赖项。
FR-03	用户可以对已生成应用继续提出修改要求，系统根据现有项目上下文增量修改代码。
FR-04	系统应提供本地预览入口，用户可以运行并查看生成结果。
FR-05	系统应保存项目配置，包括模型设置、项目路径、最近生成记录和预览状态。
FR-06	当生成失败时，系统应展示错误原因，并给出可重新执行或手动修复的建议。

5.3 非功能需求样例

编号	AI 生成需求
NFR-01	系统应优先在本地完成项目文件生成、预览和配置保存，避免不必要的数据上传。
NFR-02	对文件写入、依赖安装和命令执行操作，应在执行前向用户展示影响范围。
NFR-03	生成过程失败后不得破坏已有项目文件，必要时应保留回滚点或变更记录。
NFR-04	系统应兼容 Windows、macOS、Linux 的基本路径和权限差异。
NFR-05	用户输入和模型输出写入项目文件前，应进行基本校验，降低提示词注入和危险命令风险。

5.4 验收标准样例

编号	AI 生成验收标准
AC-01	输入“生成一个待办事项应用”后，系统能在指定目录创建可运行项目，并包含至少一个页面和一个交互组件。
AC-02	用户要求“把待办事项增加完成状态”后，系统能在原项目基础上修改代码，而不是重新创建无关项目。
AC-03	当目标目录无写入权限时，系统应停止写入并显示可理解的错误提示。
AC-04	预览启动失败时，系统应展示失败日志摘要和重试入口。
AC-05	关闭并重新打开 Dyad 后，最近项目路径和模型配置仍可恢复。

5.5 一致性检查结果

AI 识别出 4 类问题：

类型	检查结果	处理建议
约束冲突	“本地运行”与“依赖云端模型服务”可能冲突	区分本地工程操作和外部模型调用，文档中写清数据边界
不可测试表述	“界面美观”“生成效果好”不可直接验收	转化为交互步骤、错误提示、响应时间和用户确认机制
范围膨胀	数据库集成、导入导出、自动发布可能超出 MVP	MVP 只保留应用生成、本地预览、增量修改和配置保存
安全遗漏	原始需求没有明确文件写入和命令执行的权限边界	增加执行前确认、变更记录和危险命令拦截

这些输出中，非功能需求和一致性检查最有价值。人工整理需求时，通常会优先写功能流程，容易忽略失败恢复、路径权限和提示词注入。AI 在这部分给出了可直接进入评审的候选条目。

6. 传统人工方式与 AI 辅助方式对比

为了让对比更具体，本次采用同一输入材料、同一输出模板进行小规模对比。人工方式按照“阅读背景 -> 手工提取功能点 -> 补充非功能需求 -> 编写验收标准 -> 自查一致性”的流程完成。AI 辅助方式按照“提交 prompt -> 生成需求草案 -> AI 检查一致性 -> 人工审查”的流程完成。

6.1 效率对比

指标	传统人工方式	AI 辅助方式	差异
需求初稿生成时间	约 28 分钟	约 3 分 10 秒	减少约 88.7%
加入人工审查后的总时间	约 35 分钟	约 9 分 10 秒	减少约 73.8%
功能需求数量	8 条	10 条，审查后保留 9 条	AI 多 1 条有效需求
非功能需求数量	3 条	7 条，审查后保留 6 条	AI 多 3 条有效需求
验收标准数量	5 条	9 条，审查后保留 8 条	AI 多 3 条有效标准

一致性问题发现数	1 个	4 个，审查后确认 3 个	AI 多发现 2 个有效问题
----------	-----	---------------	----------------

从时间看，AI 对初稿生成的提升最明显。人工方式需要先理解材料，再组织结构和措辞；AI 可以直接按模板生成一版完整草案。加入人工审查后，AI 辅助方式仍然更快，因为审查一份结构化草案比从零写文档省时。

6.2 质量对比

质量评分采用 5 个维度，每项 20 分，总分 100 分。评分依据是输出是否完整、是否可测试、是否能支持后续设计和测试。

维度	传统人工方式	AI 辅助方式	说明
完整性	15/20	18/20	AI 对非功能需求覆盖更广
可测试性	14/20	17/20	AI 生成的 AC 编号清晰，便于转测试
一致性检查	10/20	16/20	AI 更容易列出冲突和范围风险
可追踪性	15/20	15/20	两者都能追踪到项目背景，但 AI 仍需人工核对
可维护性	14/20	16/20	AI 输出结构更统一，后续可直接进入文档库
总分	68/100	82/100	AI 辅助方式质量更高，但不是免审查结果

AI 输出的主要优势在于结构完整和覆盖面广。它能快速补出安全、权限、失败恢复、跨平台兼容等条目。人工输出的优势是更贴近课程上下文，不容易加入暂时不需要的功能。比如 AI 初稿中曾建议加入“自动发布到云平台”，这与 Dyad 当前 MVP 的“发布准备”不完全一致，人工审查后应删除或降级为后续版本需求。

7. 发现的问题

本次验证也暴露了几个问题。

第一，AI 容易扩展需求范围。只要项目背景中出现“发布准备”“数据库集成”等词，模型就可能把它们写成功能需求。解决方法是在 prompt 中明确 MVP 范围，并要求模型标注“当前版本”和“后续版本”。

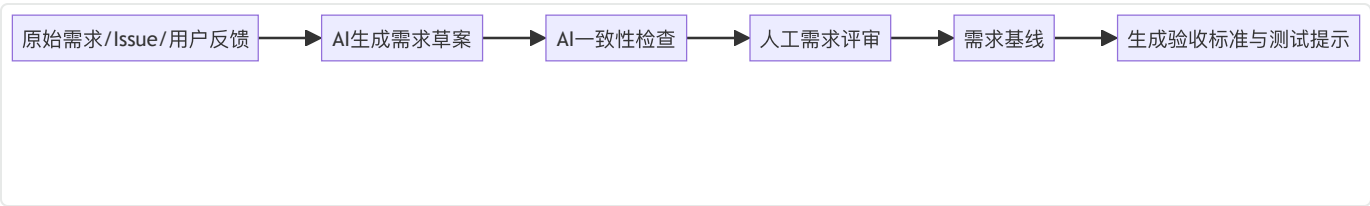
第二，AI 生成的验收标准有时过于理想。例如“生成项目应可运行”需要进一步说明运行环境、依赖安装是否成功、失败时如何判断。后续应要求 AI 输出更细的前置条件和观察结果。

第三，AI 对数据安全的提醒有帮助，但表述仍偏原则。真正落地时，需要把“危险命令拦截”“执行前确认”“日志脱敏”转化为代码级检查或测试用例。

第四，质量评分仍有人工主观性。本次评分适合作为课堂原型验证，不适合直接作为正式项目绩效指标。若进入真实团队流程，应让两名以上评审者分别评分，并记录差异。

8. 改进建议

基于本次验证，建议在 Dyad 的需求过程中正式加入 AI 辅助节点，流程如下：



具体改进建议如下：

1. 建立固定 prompt 模板。模板中必须包含项目背景、MVP 范围、输出格式、不可编造业务功能、验收标准必须可测试等约束。
2. 将 AI 需求输出保存到版本库。建议放在 `docs/requirements/` 目录，文件名包含日期和需求编号。
3. 增加需求评审清单。评审人检查来源追踪、范围是否膨胀、非功能需求是否遗漏、验收标准是否可测试。
4. 将验收标准传递给测试阶段。AI 生成的 AC 条目可以直接作为后续 Vitest 或 Playwright 测试生成的输入。
5. 建立小规模指标跟踪。每次需求整理记录耗时、需求条数、人工删除条数、冲突发现数和最终采纳率。

9. 结论

本次最小可行验证表明，使用 Codex 生成 Dyad 的轻量需求规格具有实际价值。在同一输入材料下，AI 辅助方式将需求文档总耗时从约 35 分钟降低到约 9 分 10 秒，时间减少约 73.8%；质量评分从 68

分提升到 82 分，主要提升来自非功能需求覆盖、验收标准生成和一致性检查。

不过，AI 输出不能直接作为最终需求基线。它会补充合理内容，也会引入范围膨胀和过度理想化的验收标准。更稳妥的做法是把 AI 定位为“需求草案生成器”和“一致性检查助手”，由需求负责人完成最后确认。按照这个方式，AI 可以显著减少文档起草时间，也能让 Dyad 的轻量过程保留必要的需求质量控制。